

ITEC 4366 Advanced Web Development

Lab 2: PHP Templating and Reusable Components

Purpose	Convert the Lab 1 single page site into a small multi page PHP website using reusable template components for the shared header, navigation, footer, and layout structure.
Estimated Effort	3 to 5 hours, depending on how much Lab 1 cleanup and page content expansion is needed.
Submission Format	One compressed .zip or .rar file submitted to the D2L assignment dropbox.
Folder Name	LastNameLab2, such as StinesLab2.
Point Value	25 points

Background

In Lab 1, you created a single page website to refresh your HTML, CSS, responsive design, and local development skills. In this lab, you will convert that starter website into a small PHP powered multi-page site using template chopping techniques. The goal is to separate repeated layout pieces into reusable components so that your site is easier to maintain as it grows.

This is a simple version of a pattern used across modern web development. Frameworks such as Laravel, WordPress, React, Vue, and ASP.NET MVC all rely on the same core idea: separate repeated structure from page-specific content. For this lab, PHP include files will help you practice that component mindset without needing a full framework yet.

Learning Goals

- Explain the basic difference between client-side code and server-side code.
- Use PHP include or require statements to assemble a webpage from reusable pieces.
- Create shared header, navigation, and footer components for a small website.
- Use simple PHP variables to customize page titles and page-specific information.
- Apply a consistent layout and navigation structure across multiple pages.
- Validate, test, and package a small PHP website for submission.

Starting Point

- Begin by copying your completed Lab 1 folder into /xampp/htdocs.
- Rename the copied folder using this pattern: LastNameLab2. For example: StinesLab2.
- Your Lab 2 website should still run locally through Apache using a URL similar to `http://localhost/StinesLab2/`.
- Do not submit only screenshots. Your ZIP or RAR file must include the actual website files.

Task 1: Confirm Your Local PHP Environment

- Start Apache in XAMPP or your approved local server environment.
- MySQL may be used later in the course, but this lab only requires Apache/PHP to run.

- Open your copied Lab 2 folder through localhost in the browser.
- Take a screenshot showing your website running from localhost with the browser URL visible.
- Place the screenshot inside your Lab 2 folder before compressing the final submission.

Task 2: Create Reusable Template Files

Create the following files inside your Lab 2 folder. You may create additional component files if your design benefits from them.

Required File	Purpose
index.php	Homepage with page-specific content and variables.
_header.php	Shared opening HTML, head section, page title logic, header area, and navigation if appropriate.
_footer.php	Shared footer content, closing layout elements, scripts, and closing body/html tags.
style.css	Custom CSS used to refine the visual design and override framework styles when needed.

- Move the repeated opening structure from index.php into _header.php. This usually includes the document type, html tag, head section, linked CSS, body opening, header, and navigation.
- Move the repeated closing structure from index.php into _footer.php. This usually includes footer content, closing layout tags, script links, and closing body/html tags.
- Use PHP include or require to recombine the page. Either is acceptable, but require_once is a good habit when a file is essential to the page loading correctly.
- Your page should still display correctly when viewed through localhost after the template files are created.

Example Include Pattern

The exact structure may vary based on your design, but your pages should follow a pattern similar to this:

```
<?php
$pageTitle = 'Home';
require_once '_header.php';
?>

<main>
    <!-- Page-specific content goes here. -->
</main>

<?php require_once '_footer.php'; ?>
```

Task 3: Use Page-Specific Variables

- Create a PHP variable such as \$pageTitle or \$title before including _header.php on each page.
- Use that variable inside the title tag so each page has a unique browser title.
- You may also use variables such as \$activePage, \$pageHeading, or \$pageDescription to help customize the header, navigation, or content on each page.
- For a cleaner and safer habit, use htmlspecialchars() when outputting variables into HTML. For example:
echo htmlspecialchars(\$pageTitle);

Task 4: Build a Multi-Page Website

- Create at least three additional pages based on your Lab 1 idea. This means your site should have at least four total pages including index.php.
- Each page must use the same _header.php and _footer.php files.
- Each page must have a unique value for the page title variable.
- Each page must include meaningful content that matches its purpose. Do not simply duplicate the same content across all pages.
- Your navigation menu must link to each page and work from every page.
- Optional but encouraged: add an active navigation state so users can tell which page they are currently viewing.

Task 5: Keep the Design Responsive and Professional

- Keep or improve the responsive layout from Lab 1 using Bootstrap 5, W3.CSS, or your own custom CSS.
- Maintain a consistent color scheme, typography choices, spacing, and layout across all pages.
- Use semantic HTML elements where appropriate, such as header, nav, main, section, article, aside, and footer.
- Make sure images have meaningful alt text unless the image is purely decorative.
- Test the site at desktop and mobile widths. Browser developer tools are fine for mobile testing.
- Avoid broken links, missing images, inconsistent page titles, and navigation that only works from one page.

Validation and Testing

- View each page through localhost and make sure the page renders correctly after PHP combines the template pieces.
- Use View Page Source in the browser to inspect the final generated HTML.
- Validate the generated HTML using the W3C Markup Validation Service: <https://validator.w3.org/>.
- The validator may not directly accept .php files. Use the rendered source from the browser and paste it into the validator direct input option.
- Fix HTML issues that are clearly under your control.
- CSS validation is encouraged as a debugging tool, but framework-related warnings are less important than clean, functional, intentional CSS.
- Take at least one screenshot showing the website running on localhost with the browser URL visible. More screenshots are welcome if they help document your work.

AI Tool Use

AI tools are allowed in this assignment as development support. You may use them to brainstorm layout ideas, explain PHP errors, generate starter examples, troubleshoot broken includes, or review code. However, you are ultimately responsible for the accuracy, functionality, security, originality, and completeness of the work you submit. You should test all AI-assisted code, adapt it to your own design, and understand how it works. Submitting code you cannot explain is not acceptable. AI can assist the developer, but it does not replace the developer.

Submission Checklist

- A compressed ZIP or RAR file named LastNameLab2.zip or LastNameLab2.rar.
- A working index.php file and at least three additional content pages.
- A shared _header.php file and a shared _footer.php file.
- A style.css file and any additional assets needed by the site.
- Navigation links that work across the full site.
- Unique page titles generated through PHP variables.
- Screenshot evidence showing the site running on localhost.
- All required files placed in the correct folder structure so the site runs after extraction.

Instructor Notes

The video walkthroughs are guides, not exact answers. Your final site should reflect your own design choices and your own understanding of how PHP templating works. The big idea is not just getting the include statement to run. The big idea is learning how reusable components make a website easier to manage, extend, and debug. That mindset carries directly into modern frameworks and professional web development work.